Copilot

Certainly! Let's break down the working of the given recursive algorithm for calculating the Fibonacci series and understand what it does:

1. The function `Fibbonacci(n)` takes an integer `n` as input.
2. If `n` is less than or equal to 1, the function returns `n`. This is the base case.
3. Otherwise (when `n` is greater than 1), the function recursively calculates the Fibonacci number for `n` by summing up the results of two recursive calls:
   - `Fibbonacci(n - 1)`: This computes the Fibonacci number for `n - 1`.
   - `Fibbonacci(n - 2)`: This computes the Fibonacci number for `n - 2`.
4. The final result is the sum of the two recursive calls.
5. The code then prints the Fibonacci number for `n = 5`.

Let's calculate the Fibonacci number for `n = 5` step by step:

- `Fibbonacci(5)` calls `Fibbonacci(4) + Fibbonacci(3)`
- `Fibbonacci(4)` calls `Fibbonacci(3) + Fibbonacci(2)`
- `Fibbonacci(3)` calls `Fibbonacci(2) + Fibbonacci(1)`
- `Fibbonacci(2)` returns 1 (base case)
- `Fibbonacci(1)` returns 1 (base case)

Now let's compute the values:

- `Fibbonacci(3)` = `Fibbonacci(2) + Fibbonacci(1)` = 1 + 1 = 2
- `Fibbonacci(4)` = `Fibbonacci(3) + Fibbonacci(2)` = 2 + 1 = 3
- `Fibbonacci(5)` = `Fibbonacci(4) + Fibbonacci(3)` = 3 + 2 = 5

Therefore, the output of `print(Fibbonacci(5))` will be 5.

This algorithm uses recursion to compute Fibonacci numbers, but it is not the most efficient approach due to repeated calculations. Dynamic programming or memoization can be used to optimize it. 😊